

LAPORAN PRATIKUM

STRUKTUR DATA

Queue

disusun Oleh:

Muhammad Yasin Habiburrahman

2511532016

Dosen Pengampu:

Dr. Wahyudi. S.T.M.T

Asisten Pratikum:

Sherly Sukmadira Putri



DEPARTEMEN INFORMATIKA FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia-Nya, sehingga kami dapat menyelesaikan tugas kelompok ini dengan baik. Shalawat serta salam semoga senantiasa tercurah kepada junjungan kita Nabi Muhammad SAW, keluarga, sahabat, serta para pengikutnya hingga akhir zaman.

Kami menyadari bahwa penyusunan tugas ini masih jauh dari sempurna, baik dari segi materi maupun penyajiannya. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari semua pihak demi kesempurnaan tugas kami di masa mendatang.

Akhir kata, kami mengucapkan terima kasih kepada dosen pembimbing yang telah memberikan arahan, serta kepada seluruh anggota kelompok yang telah bekerja sama dengan baik sehingga tugas ini dapat terselesaikan tepat waktu. Semoga tugas ini dapat bermanfaat bagi kami khususnya, dan bagi pembaca pada umumnya.

Padang, 30 April 2026

Muhammad Yasin Habiburrahman

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN.....	2
2.1 Implementasi Queue Menggunakan Array	2
2.2 Driver ArrayQueue	4
2.3 Membalikkan Urutan Data pada Queue (ReverseData_2016).....	5
2.4 Implementasi Queue Menggunakan LinkedList	6
2.5 Iterasi Queue Menggunakan Iterator.....	7
LATIHAN PRAKTIKUM.....	8
BAB III KESIMPULAN.....	11
DAFTAR PUSTAKA.....	12

BAB I

PENDAHULUAN

1.1 Latar Belakang

Queue merupakan salah satu implementasi dari Abstract Data Type (ADT) yang banyak digunakan dalam berbagai aplikasi nyata, seperti sistem antrian pelanggan, penjadwalan proses pada sistem operasi, pengiriman data pada jaringan komputer, hingga simulasi antrian pada dunia perbankan. Dengan memahami cara kerja Queue, pengembang perangkat lunak dapat merancang sistem yang mampu memproses data secara adil dan teratur sesuai waktu kedatangannya.

1.2 Tujuan

Praktikum ini memiliki beberapa tujuan, yaitu:

1. Memahami konsep ADT Queue dan prinsip kerja FIFO yang mendasarinya.
2. Mampu mengidentifikasi dan menjelaskan operasi-operasi dasar pada Queue seperti enqueue(), dequeue(), peek(), dan isEmpty().
3. Mampu mengimplementasikan ADT Queue menggunakan bahasa pemrograman Java.
4. Memahami perbedaan antara Queue dan Stack dalam hal urutan akses elemen, serta menentukan kapan masing-masing struktur data tepat untuk digunakan.

1.3 Manfaat Praktikum

Setelah mengikuti praktikum ini, praktikan diharapkan memperoleh manfaat sebagai berikut:

1. Praktikan mampu menerapkan konsep ADT Queue dalam menyelesaikan permasalahan pemrograman yang membutuhkan mekanisme antrian berbasis FIFO.
2. Praktikan memiliki pemahaman yang lebih komprehensif mengenai perbedaan dan kegunaan berbagai struktur data, khususnya Stack dan Queue, sehingga dapat memilih struktur yang tepat sesuai kebutuhan.
3. Praktikan mampu mengimplementasikan Queue secara mandiri sebagai bekal untuk mempelajari struktur data lanjutan seperti LinkedList, Tree, dan Graph.

BAB II PEMBAHASAN

2.1 Implementasi Queue Menggunakan Array

```

1  package pekan4_2511532016;
2
3  public class QueueArray_2511532016 {
4      int front_2016, rear_2016, size_2016;
5      int capacity_2016;
6      int array_2016[];
7
8      public QueueArray_2511532016(int capacity) {
9          this.capacity_2016 = capacity;
10         front_2016 = this.size_2016 = 0;
11         rear_2016 = capacity_2016 - 1;
12         array_2016 = new int[this.capacity_2016];
13     }
14
15     boolean isFull_2016(QueueArray_2511532016 queue) {
16         return (queue.size_2016 == queue.capacity_2016);
17     }
18
19     boolean isEmpty_2016(QueueArray_2511532016 queue) {
20         return (queue.size_2016 == 0);
21     }

```

Kode Program 2.1 – Potongan Kode Program QueueArray

Class ini mengimplementasikan Queue secara manual menggunakan array statis. Ada lima atribut utama: capacity sebagai ukuran maksimal array, size sebagai jumlah elemen yang sedang ada di antrian, front sebagai indeks elemen terdepan, rear sebagai indeks elemen paling belakang, dan array sebagai tempat penyimpanan datanya.

Fungsi isFull_2016() dan isEmpty_2016() merupakan fungsi yang mengecek kondisi antrian berdasarkan size. Antrian penuh jika size == capacity, dan kosong jika size == 0.

```

25 void enqueue_2016(int item_2016) {
26     if (isFull_2016(this)) {
27         return;
28     }
29     this.rear_2016 = (this.rear_2016 + 1) % this.capacity_2016;
30     this.array_2016[this.rear_2016] = item_2016;
31     this.size_2016 = this.size_2016 + 1;
32     System.out.println(item_2016 + " enqueued to queue");
33 }
34
35 int dequeue_2016() {
36     if (isEmpty_2016(this)) {
37         return Integer.MIN_VALUE;
38     }
39     int item_2016 = this.array_2016[this.front_2016];
40     this.front_2016 = (this.front_2016 + 1) % this.capacity_2016;
41     this.size_2016 = this.size_2016 - 1;
42     return item_2016;
43 }
44
45 int front_2016() {
46     if (isEmpty_2016(this)) {
47         return Integer.MIN_VALUE;
48     }
49     return this.array_2016[this.front_2016];
50 }
51
52 int rear_2016() {
53     if (isEmpty_2016(this)) {
54         return Integer.MIN_VALUE;
55     }
56     return this.array_2016[this.rear_2016];
57 }
58
59 void display_2016() {
60     if (isEmpty_2016(this)) {
61         System.out.println("\n Antrian Kosong");
62         return;
63     }
64     for (int i_2016 = front_2016; i_2016 < rear_2016; i_2016++) {
65         System.out.printf("%d <-- ", array_2016[i_2016]);
66     }
67     System.out.println();
68 }
69 }

```

Kode Program 2.2 – Sambungan dari Kode Program 2.1

Enqueue_2016() merupakan fungsi untuk memasukkan data ke dalam queue/barisan. Sebelum memasukkan elemen, dicek dulu apakah antrian penuh. Jika tidak, rear di-update dengan rumus $(rear + 1) \% capacity$ lalu elemen disimpan di array[rear], kemudian size bertambah 1. Rumus modulo di sini adalah kuncinya — ini yang membuat implementasi ini disebut Circular Queue. Ketika rear mencapai ujung array, modulo membuatnya "memutar balik" ke indeks 0, sehingga slot yang sudah dikosongkan oleh dequeue bisa dipakai kembali tanpa membuang memori.

Dequeue_2016() mengambil elemen di posisi front, lalu front digeser ke kanan dengan rumus $(front + 1) \% capacity$. Ini juga menggunakan modulo untuk alasan yang sama. Jika antrian kosong dikembalikan Integer.MIN_VALUE sebagai tanda error.

Front_2016() dan rear_2016() hanya mengintip nilai elemen terdepan dan terbelakang tanpa mengubah apapun, mirip seperti peek() pada Stack.

display_2016() mencetak isi antrian dari front sampai sebelum rear dengan format panah $\leftarrow$.

2.2 Driver ArrayQueue

```
1 package pekan4_2511532016;
2 public class QueueArrayDriver_2511532016 {
3     public static void main(String[] args) {
4         QueueArray_2511532016 queue_2016 = new QueueArray_2511532016(1000);
5         queue_2016.enqueue_2016(10);
6         queue_2016.enqueue_2016(20);
7         queue_2016.enqueue_2016(30);
8         queue_2016.enqueue_2016(40);
9         System.out.println("Item di depan " + queue_2016.front_2016());
10        System.out.println("Item paling belakang " + queue_2016.rear_2016());
11        System.out.println("Tampilan queue");
12        queue_2016.display_2016();
13        System.out.println(queue_2016.dequeue_2016() + " dihapus dari queue");
14        System.out.println("Item di depan " + queue_2016.front_2016());
15        System.out.println("Item paling belakang " + queue_2016.rear_2016());
16        System.out.println("Tampilan queue setelah satu data dihapus");
17        queue_2016.display_2016();
18    }
19 }
20 }
21 }
```

Kode Program 2.3 – Driver dari ArrayQueue sebagai Pengujian

Queue dengan kapasitas 1000 dibuat sebagai bahan uji, lalu empat elemen dimasukkan berurutan: 10, 20, 30, 40; menggunakan enqueue(). Setelah itu dicetak elemen terdepan via front(), yaitu 10, elemen terbelakang via rear(), yaitu 40, lalu seluruh isi queue ditampilkan via display().

Kemudian dequeue() dipanggil yang mengambil dan menghapus elemen terdepan yaitu 10 sesuai prinsip FIFO, sehingga front bergeser ke elemen berikutnya yaitu 20. Keluaran dari program tersebut setelah dijalankan adalah sebagai berikut.

```

10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue
Item di depan 10
Item paling belakang 40
Tampilan queue
10 <-- 20 <-- 30 <--
10 dihapus dari queue
Item di depan 20
Item paling belakang 40
Tampilan queue setelah satu data dihapus
20 <-- 30 <--

```

Gambar 2.1 – Hasil dari Pengujian Queue Array

2.3 Membalikkan Urutan Data pada Queue (ReverseData_2016)

```

1 package pekan4_2511532016;
2 import java.util.Queue;
3 import java.util.Stack;
4 import java.util.LinkedList;
5 public class ReverseData_2511532016 {
6     public static void main(String[] args) {
7         Queue<Integer> q_2016 = new LinkedList<Integer>();
8         q_2016.add(1);
9         q_2016.add(2);
10        q_2016.add(3);
11        System.out.println("Sebelum reverse " + q_2016);
12        Stack<Integer> s_2016 = new Stack<Integer>();
13        while(!q_2016.isEmpty()) {
14            s_2016.push(q_2016.remove());
15        }
16        while(!s_2016.isEmpty()) {
17            q_2016.add(s_2016.pop());
18        }
19        System.out.println("Sesudah reverse= " + q_2016);
20    }
21 }

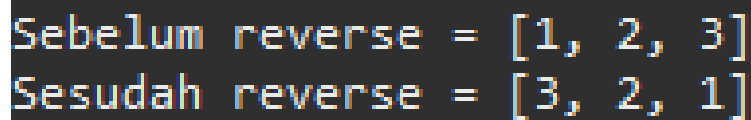
```

Kode Program 2.4 – Kode Program untuk Membalikkan Urutan Data

Program ini mendemonstrasikan cara membalik urutan elemen dalam Queue menggunakan Stack sebagai perantara.

Pertama, tiga elemen 1, 2, 3 dimasukkan ke Queue sehingga urutan awalnya adalah [1, 2, 3] dengan 1 di posisi terdepan. Kemudian, semua elemen di-`remove()` dari Queue satu per satu dan di-`push()` ke Stack. Karena Stack bersifat LIFO, urutan di Stack menjadi 3 di atas, lalu 2, lalu 1 di paling bawah.

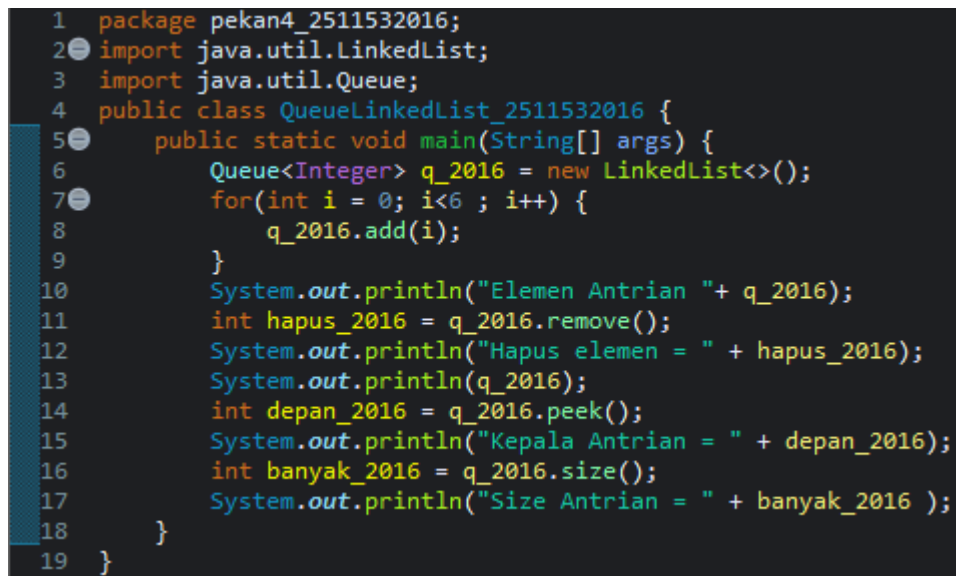
Pada loop kedua, semua elemen di-`pop()` dari Stack dan di-`add()` kembali ke Queue. Karena yang keluar duluan dari Stack adalah 3, maka Queue terisi dengan urutan [3, 2, 1], berhasil terbalik. Keluaran dari program sesuai dengan penjelasan.



```
Sebelum reverse = [1, 2, 3]
Sesudah reverse = [3, 2, 1]
```

Gambar 2.2 – Hasil Keluaran dari Program Reverse Data

2.4 Implementasi Queue Menggunakan LinkedList



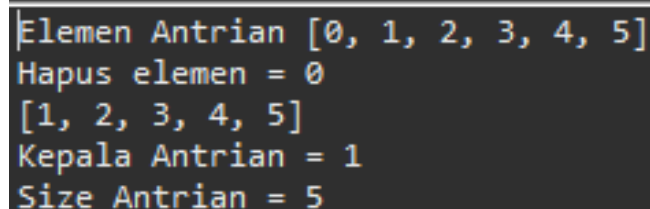
```
1 package pekan4_2511532016;
2 import java.util.LinkedList;
3 import java.util.Queue;
4 public class QueueLinkedList_2511532016 {
5     public static void main(String[] args) {
6         Queue<Integer> q_2016 = new LinkedList<>();
7         for(int i = 0; i<6 ; i++) {
8             q_2016.add(i);
9         }
10        System.out.println("Elemen Antrian "+ q_2016);
11        int hapus_2016 = q_2016.remove();
12        System.out.println("Hapus elemen = " + hapus_2016);
13        System.out.println(q_2016);
14        int depan_2016 = q_2016.peek();
15        System.out.println("Kepala Antrian = " + depan_2016);
16        int banyak_2016 = q_2016.size();
17        System.out.println("Size Antrian = " + banyak_2016 );
18    }
19 }
```

Kode Program 2.5 – Kode Program Implementasi Queue Menggunakan LinkedList

Program ini mendeklarasikan Queue bertipe Integer menggunakan LinkedList sebagai implementasinya, lalu mengisi antrian dengan angka 0 sampai 5 menggunakan perulangan for sehingga isi awalnya adalah [0, 1, 2, 3, 4, 5].

Kemudian `remove()` dipanggil yang mengambil dan menghapus elemen terdepan yaitu 0, nilai yang dihapus ditampung ke variabel `hapus` dan dicetak.

Setelah itu `peek()` digunakan untuk mengintip elemen terdepan yang sekarang adalah 1 tanpa menghapusnya, dan terakhir `size()` mencetak jumlah elemen yang tersisa yaitu 5. Keluarannya adalah sebagai berikut.



```
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus elemen = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5
```

Gambar 2.3 – Hasil Keluaran Program Queue dengan Implementasi LinkedList

2.5 Iterasi Queue Menggunakan Iterator

```
1 package pekan4_2511532016;
2 import java.util.Queue;
3 import java.util.Iterator;
4 import java.util.LinkedList;
5 public class IterasiQueue_2511532016 {
6     public static void main(String[] args) {
7         Queue<String> q_2016 = new LinkedList<>();
8         q_2016.add("Praktikum");
9         q_2016.add("Struktur");
10        q_2016.add("Data");
11        q_2016.add("Dan");
12        q_2016.add("Algoritma");
13        Iterator<String> iterator = q_2016.iterator();
14        while(iterator.hasNext()){
15            System.out.print(iterator.next() + " ");
16        }
17    }
18 }
```

Kode Program 2.6 – Program Iterasi Queue Menggunakan Iterator

Iterator adalah objek yang memungkinkan traversal elemen koleksi satu per satu tanpa perlu tahu struktur internalnya. Dua method utamanya:

hasNext() : Cek apakah masih ada elemen berikutnya

next() : Ambil elemen saat ini dan maju ke berikutnya

Program ini membuat Queue bertipe String menggunakan LinkedList, lalu mengisi antrian dengan lima kata, yaitu "Praktikum", "Struktur", "Data", "Dan", "Algoritma".

Untuk mencetak isinya, digunakan Iterator yang diperoleh dari method iterator() milik Queue. Loop while berjalan selama hasNext() mengembalikan true artinya masih ada elemen berikutnya dan tiap iterasi next() dipanggil untuk mengambil dan berpindah ke elemen selanjutnya.

LATIHAN PRAKTIKUM

```
1 package pekan4_2511532016;
2
3 public class Queue_2511532016 {
4     int front_2016, rear_2016, size_2016, max_2016;
5     String array_2016[];
6     public Queue_2511532016(int max_2016) {
7         this.max_2016 = max_2016;
8         front_2016 = this.size_2016 = 0;
9         rear_2016 = max_2016 - 1;
10        array_2016 = new String[this.max_2016];
11    }
12
13    void enqueue_2016(String anu_2016) {
14        if (isFull_2016(this)) {
15            return;
16        }
17        this.rear_2016 = (this.rear_2016 + 1) % this.max_2016;
18        this.array_2016[this.rear_2016] = anu_2016;
19        this.size_2016 = this.size_2016 + 1;
20        System.out.println(anu_2016 + " berhasil masuk ke antrian");
21    }
22
23    void dequeue_2016() {
24        if (isEmpty_2016(this)) {
25            return;
26        }
27        String item_2016 = this.array_2016[this.front_2016];
28        this.front_2016 = (this.front_2016 + 1) % this.max_2016;
29        this.size_2016 = this.size_2016 - 1;
30        System.out.println(item_2016 + " telah dilayani");
31    }
}
```

Potongan Kode Program Latihan ADT Pekan 3 – Bagian 1

Kode program di atas menampilkan method berupa konstruktor untuk membuat Queue. enqueue_2016 untuk memasukkan data berupa String ke dalam Queue, dan dequeue untuk menghapus elemen antrian paling depan,

```

34 void display_2016() {
35     if (isEmpty_2016(this)) {
36         return;
37     }
38     System.out.println("Isi antrian:");
39     for (int i_2016 = 0; i_2016 < this.size_2016; i_2016++) {
40         System.out.println((i_2016 + 1) + ". " + array_2016[(front_2016 + i_2016) % max_2016]);
41     }
42 }
43
44 boolean isEmpty_2016(Queue_2511532016 queue_2016) {
45     if (queue_2016.size_2016 == 0) {
46         System.out.println("Antrian Kosong!");
47         return true;
48     }
49     return false;
50 }
51
52 boolean isFull_2016(Queue_2511532016 queue) {
53     if (queue.size_2016 == queue.max_2016) {
54         System.out.println("Antrian Sudah Penuh!");
55         return true;
56     }
57     return false;
58 }
59 void Reverse_2016() {
60     if (isEmpty_2016(this)) {
61         return;
62     }
63     String[] temp_2016 = new String[this.size_2016];
64     for (int i_2016 = 0; i_2016 < this.size_2016; i_2016++) {
65         temp_2016[i_2016] = this.array_2016[(front_2016 + i_2016) % this.max_2016];
66     }
67     for (int i_2016 = 0; i_2016 < this.size_2016; i_2016++) {
68         this.array_2016[i_2016] = temp_2016[this.size_2016 - 1 - i_2016];
69     }
70     this.front_2016 = 0;
71     this.rear_2016 = this.size_2016 - 1;
72     this.display_2016();
73
74     System.out.println();
75 }
76 }

```

Potongan Kode Program Latihan ADT Pekan 3 – Bagian 2

Method display_2016 digunakan untuk menampilkan isi dari antrian. Kemudian, isEmpty_2016 dan isFull_2016 digunakan untuk mengecek apakah antrian kosong, ataupun penuh. Method terakhir adalah method pembalik barisan.

```

1 package pekan4_2511532016;
2 import java.util.Scanner;
3 public class AntrianLoket_2511532016 {
4     public static void tampilkanMenu_2511532016() {
5         System.out.println("\nMenu:");
6         System.out.println("1. Tambah Antrian");
7         System.out.println("2. Hapus Antrian");
8         System.out.println("3. Tampilkan Antrian");
9         System.out.println("4. Reverse Antrian");
10        System.out.println("5. Keluar");
11    }
12    public static void main(String[] args) {
13        Queue_2511532016 Antrian = new Queue_2511532016(100);
14        Scanner scanner = new Scanner(System.in);
15        int choice;
16        do {
17            tampilkanMenu_2511532016();
18            System.out.print("Pilih menu: ");
19            choice = scanner.nextInt();
20            scanner.nextLine();
21            switch(choice){
22                case 1:
23                    System.out.print("Masukkan nama pelanggan: ");
24                    String nama = scanner.nextLine();
25                    Antrian.enqueue_2016(nama);
26                    break;
27                case 2:
28                    Antrian.dequeue_2016();
29                    break;
30                case 3:
31                    Antrian.display_2016();
32                    break;
33                case 4:
34                    Antrian.Reverse_2016();
35                    break;
36                case 5:
37                    System.out.println("Keluar dari program.");
38                    break;
39                default:
40                    System.out.println("Pilihan tidak valid!");
41                    break;
42            }
43        } while (choice != 5);
44        scanner.close();
45    }
46 }
47
48
49
50

```

```

<terminated> AntrianLoket_2511532016
Masukkan nama pelanggan: Andi
Andi berhasil masuk ke antrian

Menu:
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse Antrian
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: budi
budi berhasil masuk ke antrian

Menu:
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse Antrian
5. Keluar
Pilih menu: 3
Isi antrian:
1. Andi
2. budi

Menu:
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse Antrian
5. Keluar
Pilih menu: 4
Isi antrian:
1. budi
2. Andi

Menu:
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse Antrian
5. Keluar
Pilih menu: 2
budi telah dilayani

Menu:
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse Antrian
5. Keluar
Pilih menu: 3
Isi antrian:
1. Andi

Menu:
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian

```

Program Driver Latihan Praktikum Pekan 3

BAB III

KESIMPULAN

Queue adalah struktur data yang bekerja berdasarkan prinsip FIFO (First In, First Out), di mana elemen yang pertama kali dimasukkan akan menjadi elemen yang pertama kali dikeluarkan. Operasi dasar Queue meliputi enqueue() untuk memasukkan elemen ke belakang antrian, dequeue() untuk mengeluarkan elemen dari depan antrian, display() untuk menampilkan seluruh isi antrian, serta isEmpty() dan isFull() untuk mengecek kondisi antrian.

Praktikum ini menunjukkan bahwa Queue dapat diimplementasikan dengan berbagai pendekatan — menggunakan array statis secara manual, menggunakan LinkedList sebagai implementasi interface Queue bawaan Java, maupun dikombinasikan dengan Stack untuk keperluan tertentu. Pada implementasi array manual, digunakan teknik Circular Queue dengan operasi modulo pada enqueue dan dequeue sehingga slot memori yang sudah dikosongkan dapat digunakan kembali tanpa perlu menggeser seluruh elemen array.

Selain itu, praktikum ini juga mendemonstrasikan hubungan berlawanan antara Queue dan Stack — dengan memindahkan elemen Queue ke Stack lalu mengembalikannya, urutan elemen dapat dibalik karena sifat LIFO pada Stack bertolak belakang dengan sifat FIFO pada Queue. Secara keseluruhan, pemahaman terhadap ADT Queue melengkapi pemahaman terhadap Stack dan ArrayList, serta memberikan bekal yang kuat untuk mempelajari struktur data lanjutan.

DAFTAR PUSTAKA

- [1]Oracle. *Java Documentation*. <https://docs.oracle.com/javase/>